CSE 4309: Theory of Computing

Lecture 1 — Introduction, Finite Automata, Regular Expressions

Syed Rifat Raiyan
Lecturer
Department of Computer Science & Engineering
Islamic University of Technology, Dhaka, Bangladesh
Email: rifatraiyan@iut-dhaka.edu

Introduction

First things first…

Google Classroom – yzh4o7x7

Relevant Resources

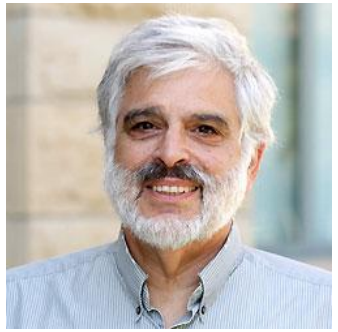
- Books:
  - ✓ ***Introduction to the Theory of Computation*** by Michael Sipser

    Link: Sipser 3rd Edition
  - ✓ ***Introduction to Automata Theory, Languages, and Computation*** by Ullman and Hopcroft

collect from library

- Video Lectures — YouTube channel ***MIT OpenCourseWare***

  Link: 18.404J Fall 2020 Lectures



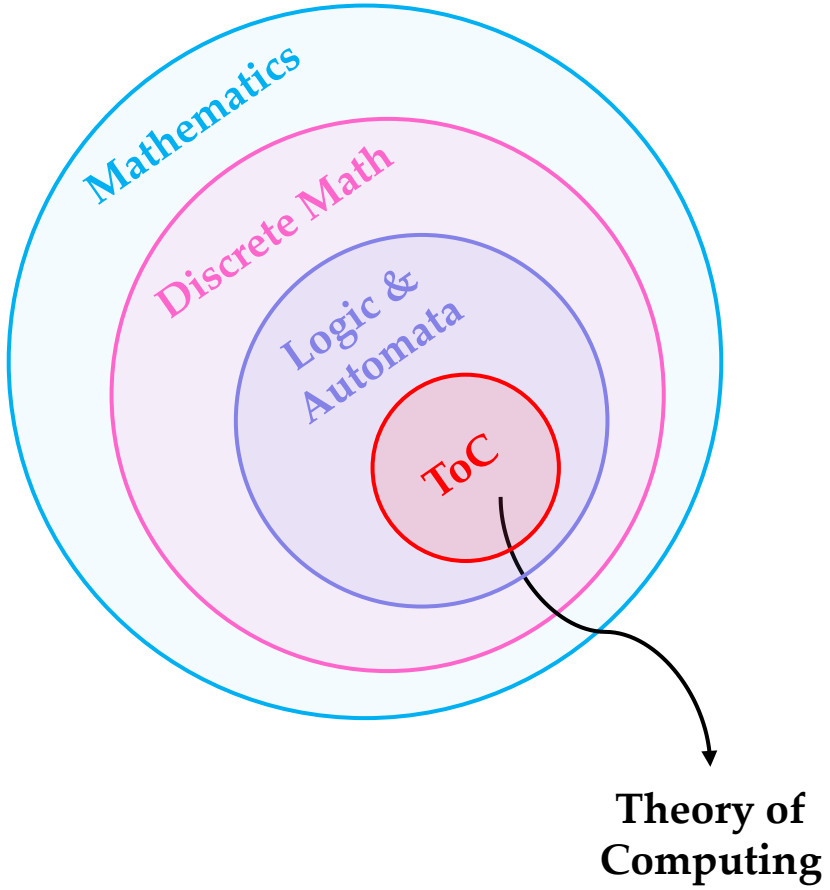
Keep an eye on the GC for the Slides and Lecture notes.

Introduction

How to do well in this course?

For the **theory course** CSE 4309 —

- Pay attention to the class lectures.
- Practise examples from:
  - ✓ Lecture slides
  - ✓ Sipser's book
  - ✓ Hopcroft's book
- Solve previous-year questions (problems and proofs).
- Practise, practise, and practise…



Lecture Plan

The agenda for today

- What is Theory of Computing?
- Why do we need it?
- Finite Automaton
- Regular Language
- Regular Expression (regex)
- Regular Language Property: Closure under Union

Expect some **mini quizzes**!

(for easy *bonus marks* 😉)

Theory of Computing

What it is and why do we need it?

It is a mathematical study of the ***limits*** of computation, focusing on:

- What can be computed (**Computability** Theory)
  - Can we solve any mathematical question methodically? Gödel's theorem says NO!
- How efficiently it can be computed (**Complexity** Theory)
  - Fast — polynomial time
- How computation can be modeled (**Automata** Theory)
  - Tractability of the problem given constant space

Applications:

- Cryptography — Relies on computational hardness
- Compiler Design — Uses formal languages and automata
- AI & Machine Learning — Bounded rationality and complexity limits
- Verification — Decidability of correctness proofs

Finite Automata

The *informal* definition

Automaton: A self-operating machine that performs a predetermined sequence of actions.

Plural: Automata

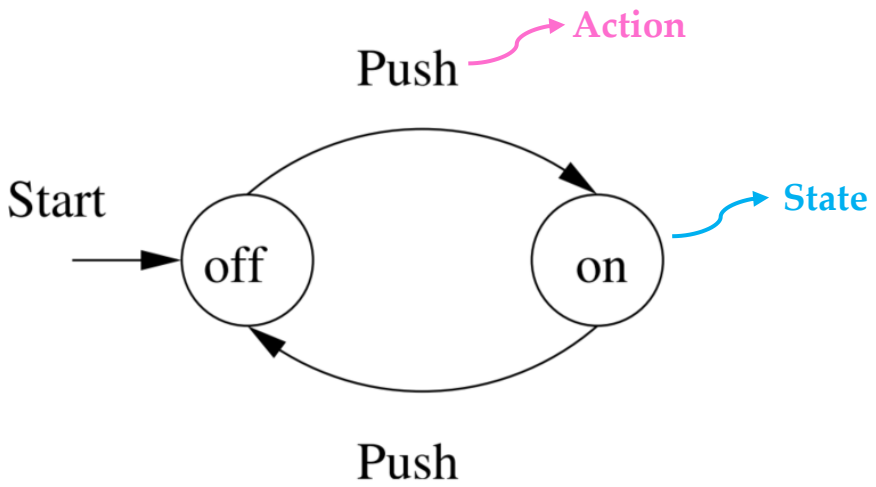
Origin: Greek word αὐτόματον

Pinocchio Animatronic Automaton

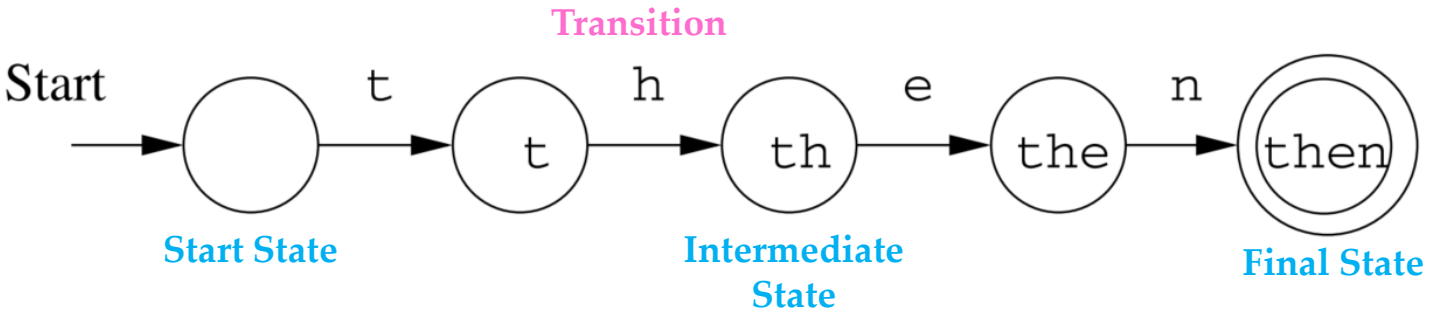
Finite Automata

Simple examples

The *simplest* non-trivial finite automaton is an on/off switch.

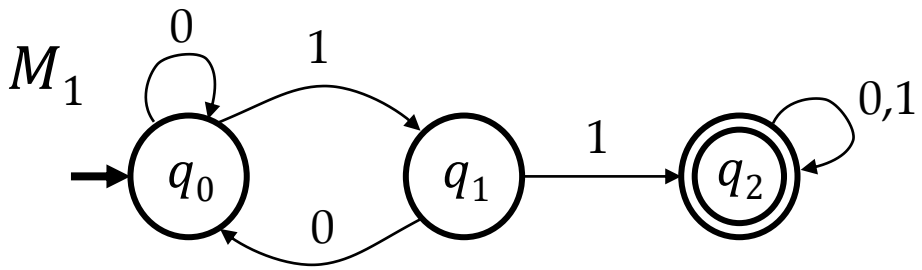


Modeling recognition of the word *"then"* in a lexical analyzer.



Finite Automata

Terminologies



States: $q_0\ q_1\ q_2$

Transitions: $\xrightarrow{1}$

Start state:

Accept states:

Input: finite string
Output: Accept or Reject
Computation process: Begin at start state, read input symbols, follow corresponding transitions, Accept if end with accept state, Reject if not.

Examples: 01101 → Accept
00101 → Reject

M_1 accepts exactly those strings in A where
$A = \{w \mid w \text{ contains substring } 11\}$.

Say that A is the language of M_1 and that M_1 recognizes A and that $A = L(M_1)$.

Finite Automata

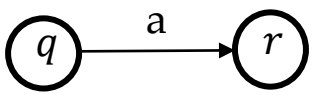
The *formal* definition

Defn: A <u>finite automaton</u> M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

Q finite set of states

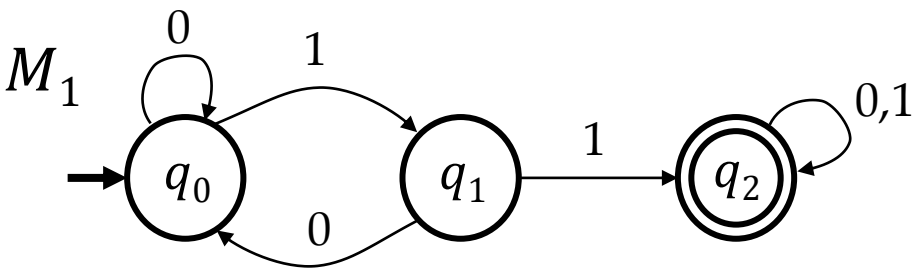
Σ finite set of alphabet symbols

δ transition function $\delta\colon Q \times \Sigma \to Q$ $\quad \delta(q, a) = r$ means



q_0 start state

F set of accept states



$M_1 = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

$F = \{q_2\}$

δ	0	1
q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_2	q_2

Finite Automata

Some more terminologies

Strings and languages

- A <u>string</u> is a finite sequence of symbols in Σ, *e.g.*, Σ = {0, 1}
- A <u>language</u> is a set of strings (finite or infinite), *e.g.*, L = {001, 11, 1101}
- The <u>empty string</u> ε is the string of length 0
- The <u>empty language</u> ø is the set with no strings

Defn: M <u>accepts string</u> $w = w_1 w_2 \ldots w_n$ each $w_i \in \Sigma$
if there is a sequence of states $r_0, r_1, r_2, , \ldots, r_n \in Q$
where:

- $r_0 = q_0$
- $r_i = \delta(r_{i-1}, w_i)$ for $1 \leq i \leq n$
- $r_n \in F$

✓ $L(M) = \{w \mid M \text{ accepts } w\}$

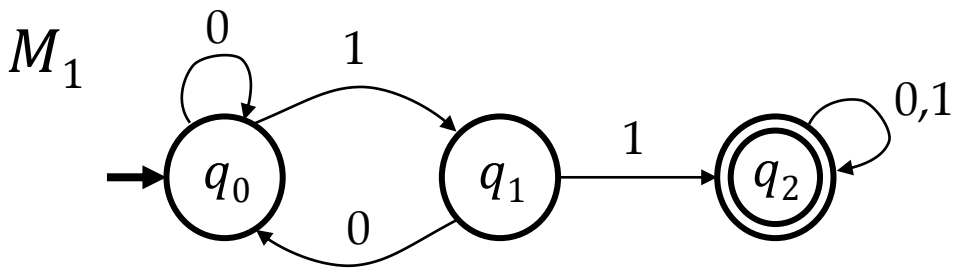
✓ $L(M)$ is <u>the language of</u> M

✓ M <u>recognizes</u> $L(M)$

Defn: A language is ***regular*** if some finite automaton recognizes it.

Regular Languages

Examples



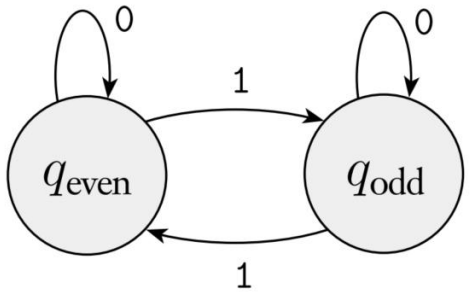
$L(M_1) = \{w \mid w \text{ contains substring } 11\} = A$

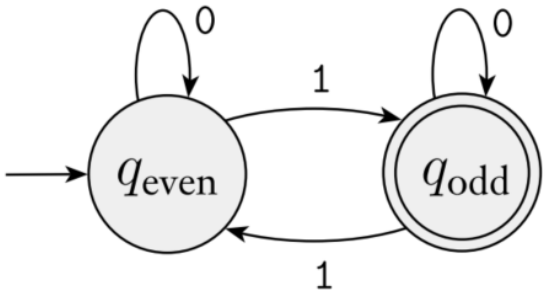
Therefore A is ***regular***.

More examples:

Let $B = \{w \mid w \text{ has an odd number of 1s}\}$

B is regular (let's make it!).





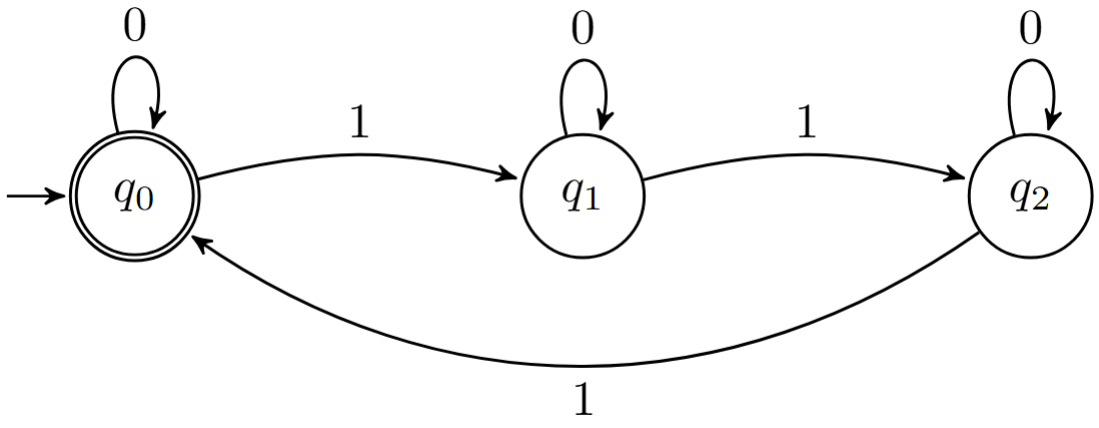
Let $C = \{w \mid w \text{ has equal numbers of 0s and 1s}\}$

C is ***not*** regular (we will prove).

Mini Quiz

Let's make another one

$$L = \{w \mid w \text{ has number of 1s} \equiv 0 \bmod 3\}$$



$$L = \{w \mid w \text{ has number of 1s} \equiv 1 \bmod 3\}$$

Regular Expressions

What are they?

Definition: Regular expression is a *compact* description of a set of strings.

Regular operations. Let A, B be languages:

- **Union:** $A \cup B = \{w \mid w \in A \text{ or } w \in B\}$
- **Concatenation:** $A \circ B = \{xy \mid x \in A \text{ and } y \in B\} = AB$
- **Star:** $A^* = \{x_1 \ldots x_k \mid \text{each } x_i \in A \text{ for } k \geq 0\}$
  Note: $\varepsilon \in A^*$ always

Example. Let $A = \{\texttt{good}, \texttt{bad}\}$ and $B = \{\texttt{boy}, \texttt{girl}\}$.

- $A \cup B = \{\texttt{good}, \texttt{bad}, \texttt{boy}, \texttt{girl}\}$
- $A \circ B = AB = \{\texttt{goodboy}, \texttt{goodgirl}, \texttt{badboy}, \texttt{badgirl}\}$
- $A^* = \{\varepsilon, \texttt{good}, \texttt{bad}, \texttt{goodgood}, \texttt{goodbad}, \texttt{badgood}, \texttt{badbad}, \texttt{goodgoodgood}, \texttt{goodgoodbad}, \ldots\}$

Regular expressions

- Built from Σ, members $\Sigma, \emptyset, \varepsilon$ **[Atomic]**
- By using $\cup, \circ, *$ **[Composite]**

Examples:

- $(0 \cup 1)^* = \Sigma^*$ gives all strings over Σ
- $\Sigma^* 1$ gives all strings that end with 1
- $\Sigma^* 11 \Sigma^*$ = all strings that contain 11 $= L(M_1)$

Goal: Show that finite automata are equivalent to regular expressions

Regular Languages

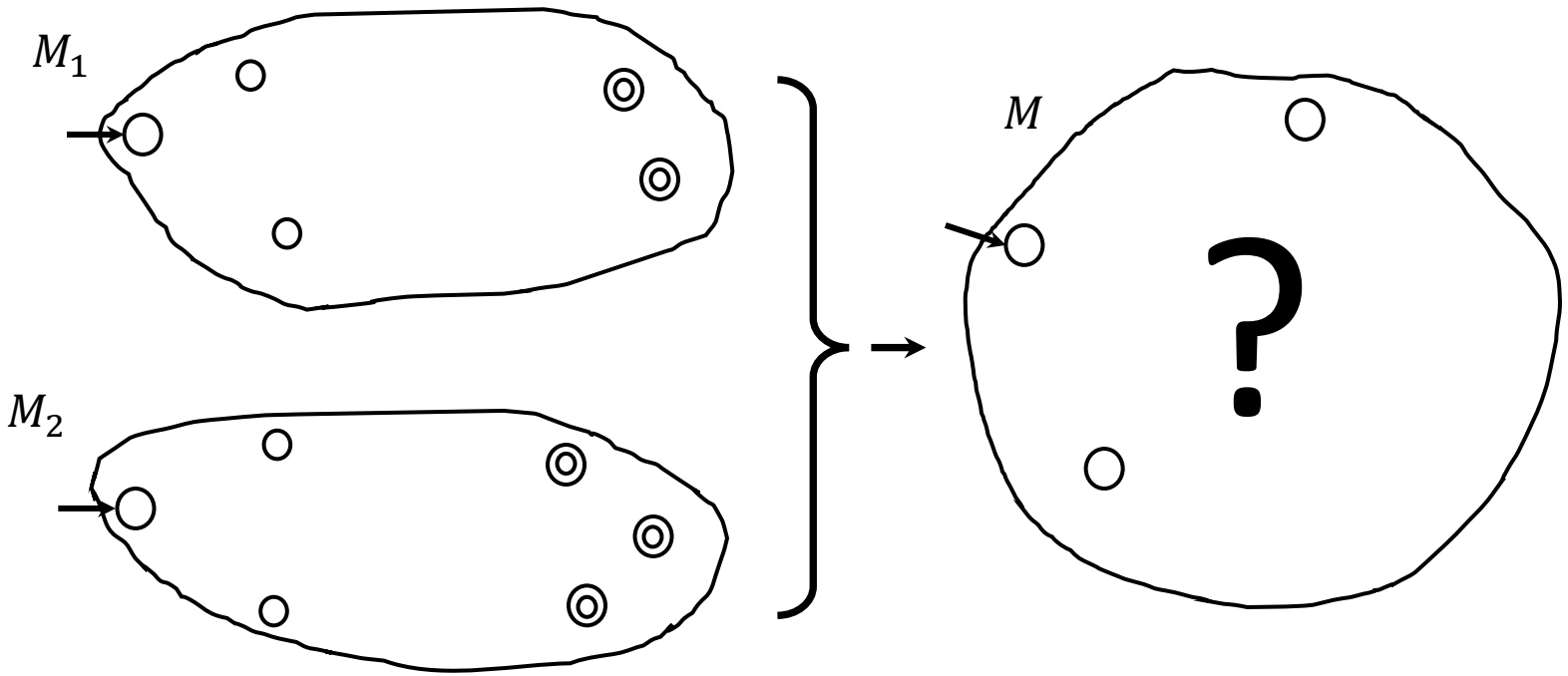
Closure Properties

Theorem: **If A_1, A_2 are regular languages, then so is $A_1 \cup A_2$.** (closure under $\cup$)

Proof: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1
$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2

Construct $M = (Q, \Sigma, \delta, q_0, F)$ recognizing $A_1 \cup A_2$

M should accept input w if either M_1 or M_2 accepts w.



Components of M:

$Q = Q_1 \times Q_2$
$= \{(q_1, q_2) \mid q_1 \in Q_1$ **and** $q_2 \in Q_2\}$

$q_0 = (q_1, q_2)$

$\delta((q, r), a) = (\delta_1(q, a), \delta_2(r, a))$

$F = \cancel{F_1 \times F_2}$ **NO! [gives intersection]**

$F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

Mini Quiz

Number of states

In the proof, if M_1 and M_2 are finite automata where M_1 has k_1 states and M_2 has k_2 states. Then how many states does M have?

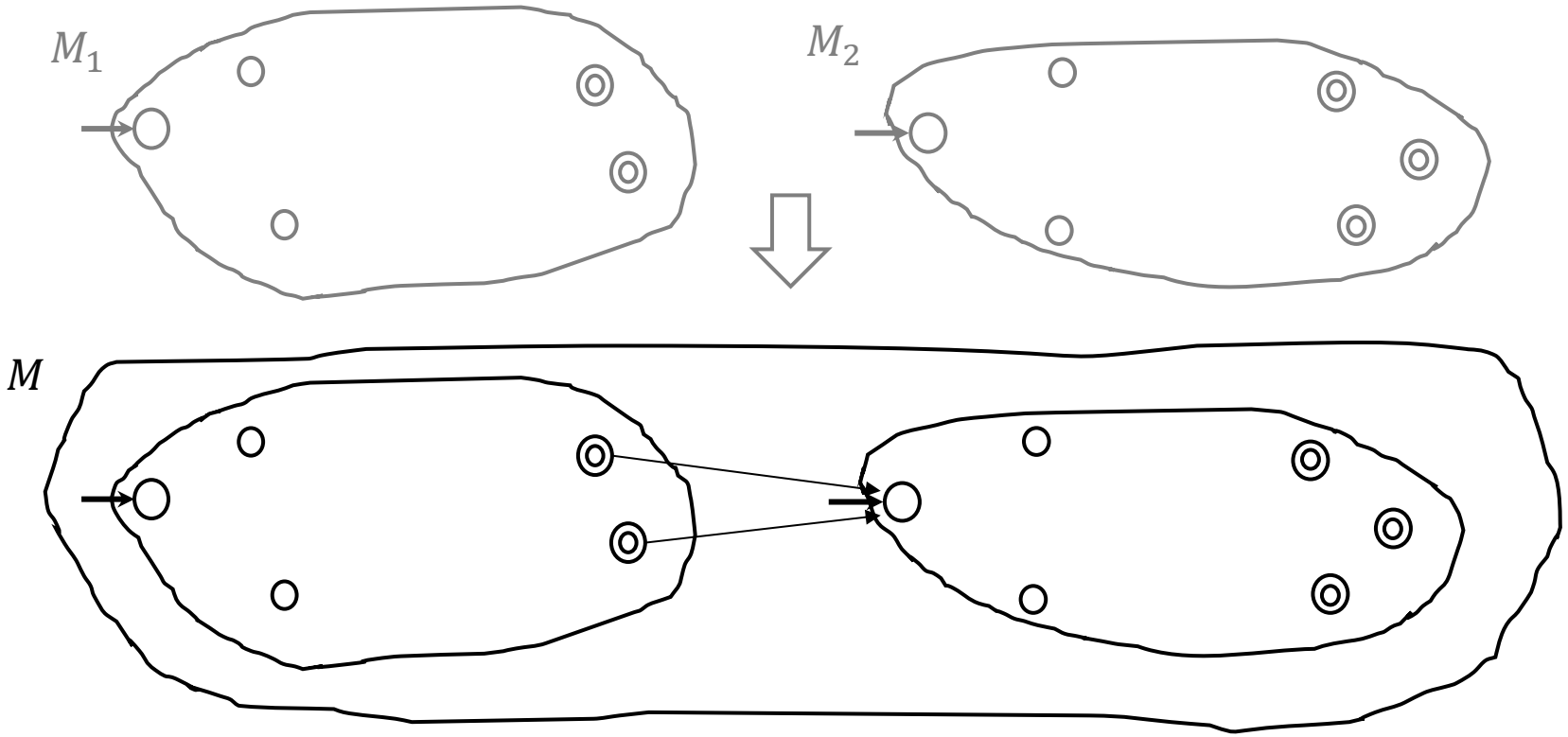
Regular Languages

Closure Properties

Theorem: **If A_1, A_2 are regular languages, then so is A_1A_2.** (closure under $\circ$)

Proof: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1
$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$

Construct $M = (Q, \Sigma, \delta, q_0, F)$



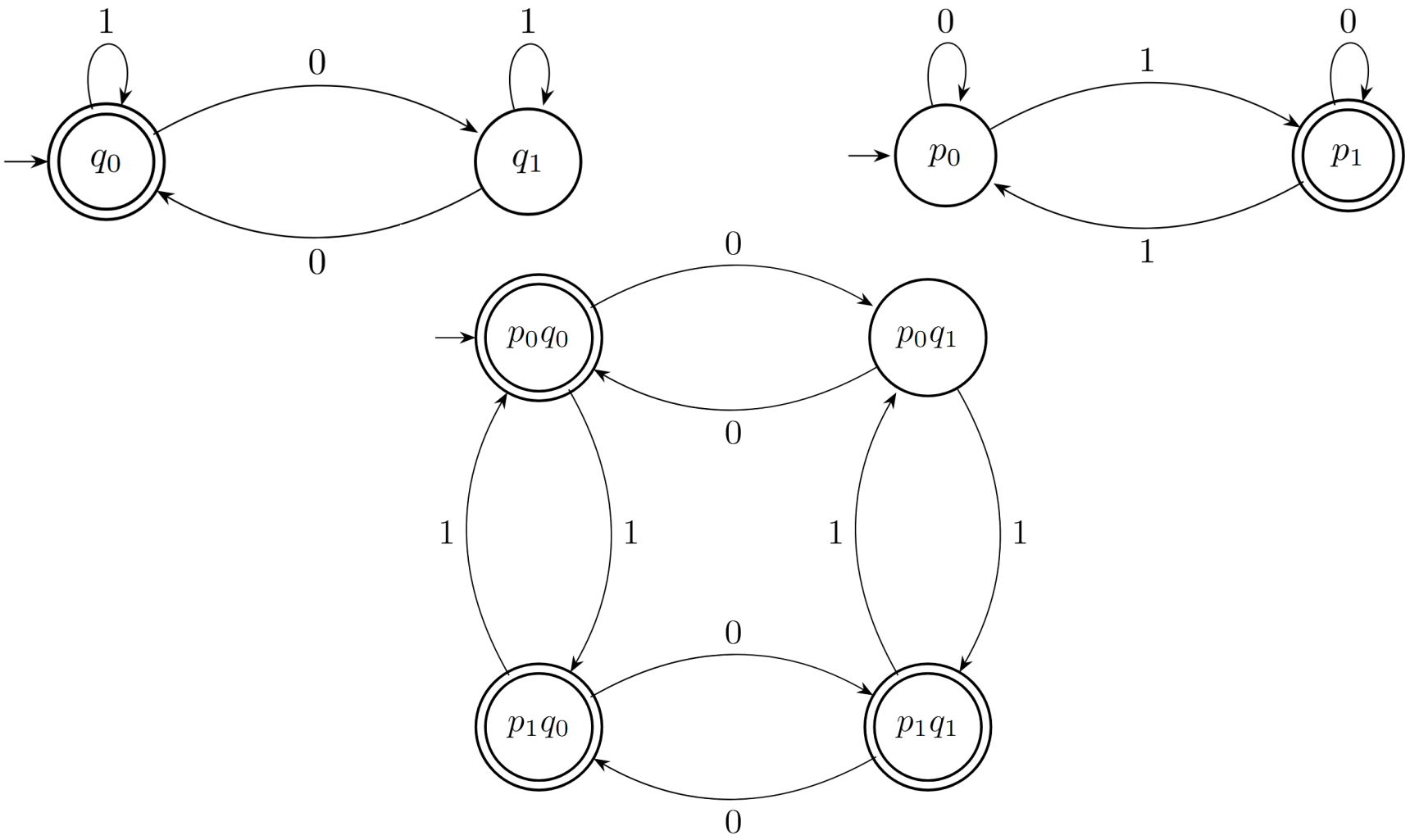
M should accept input w
if $w = xy$ where
M_1 accepts x and M_2 accepts y.

Doesn't work: Where to split w?

Examples

Let's practice!

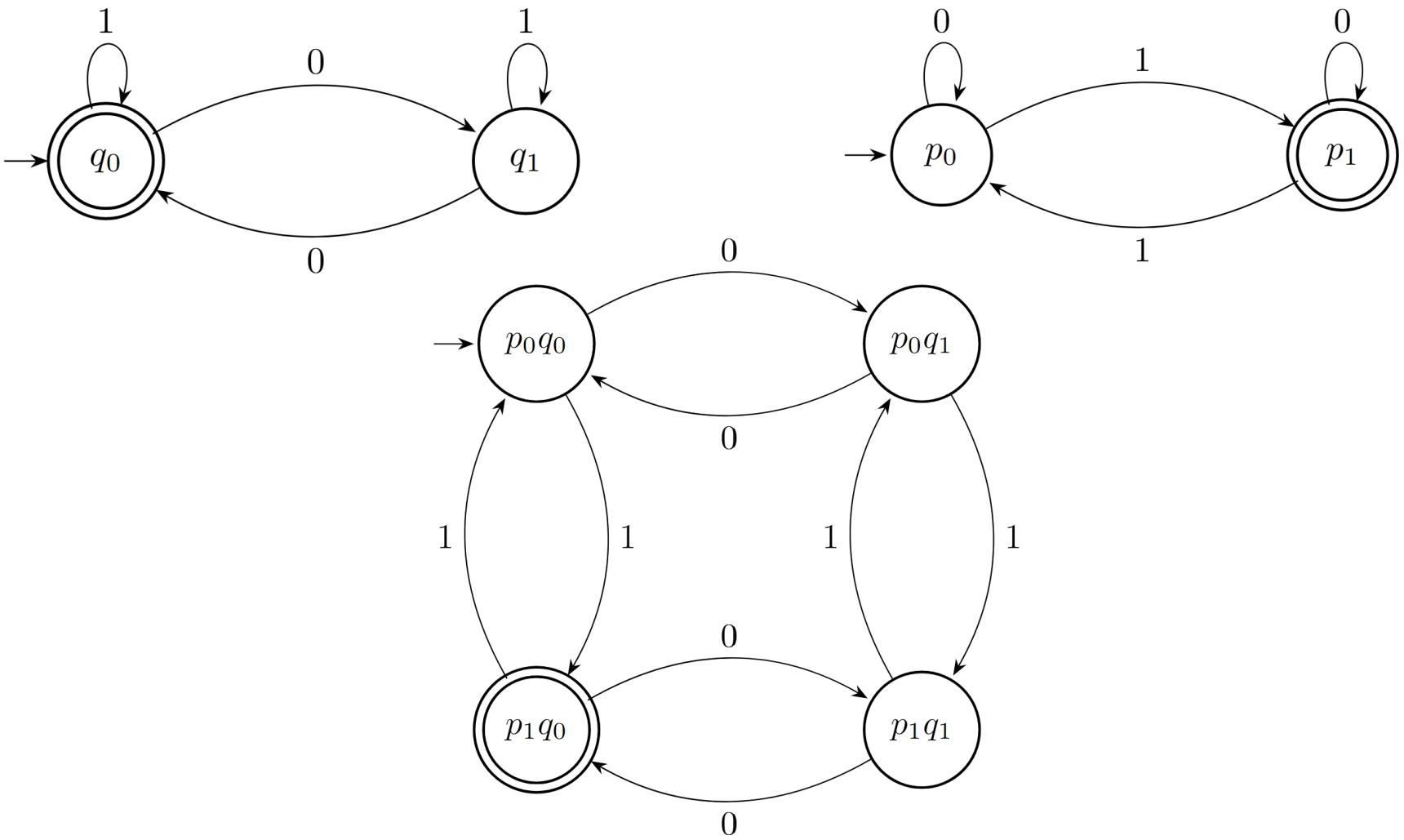
$L = \{w \mid w$ has even no. of 0s **<u>or</u>** odd no. of 1s$\}$



Examples

Let's practice!

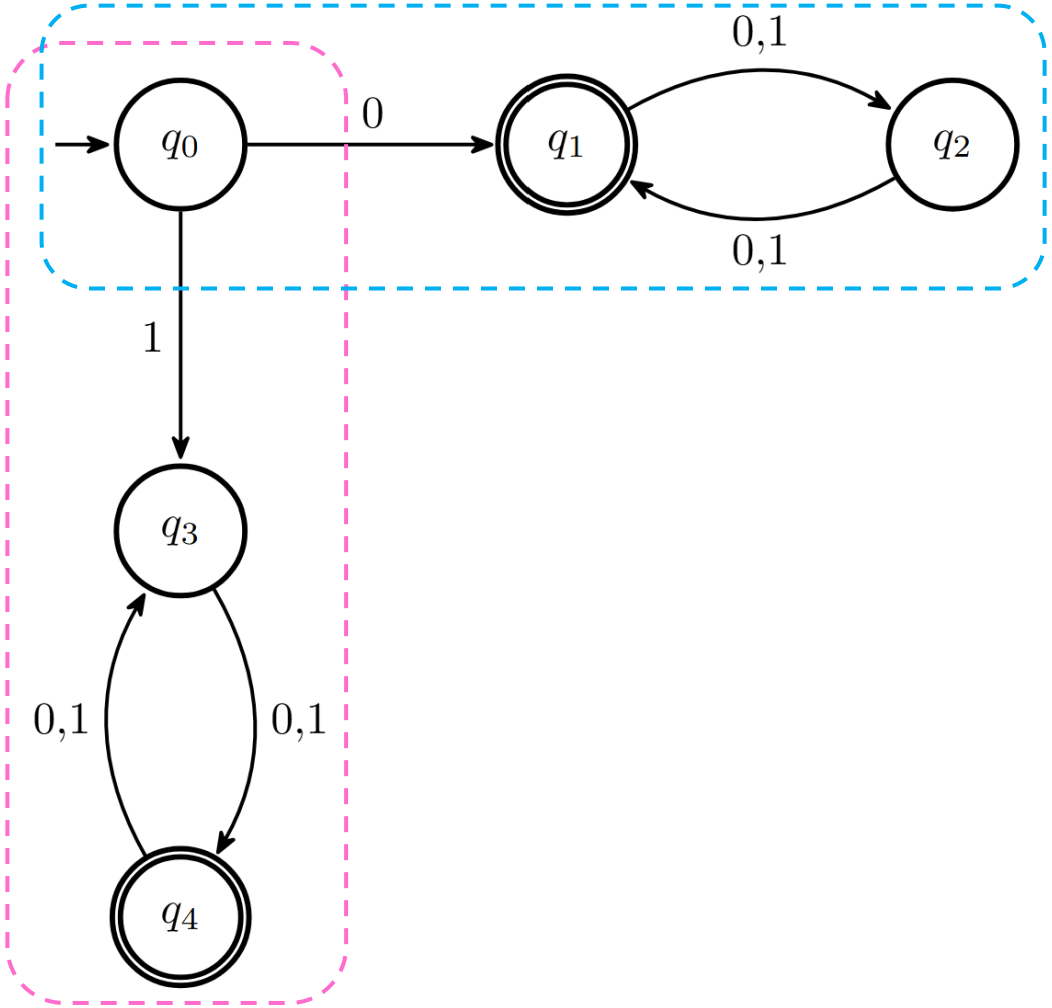
$L = \{w \mid w$ has even no. of 0s **and** odd no. of 1s$\}$



Examples

Let's practice!

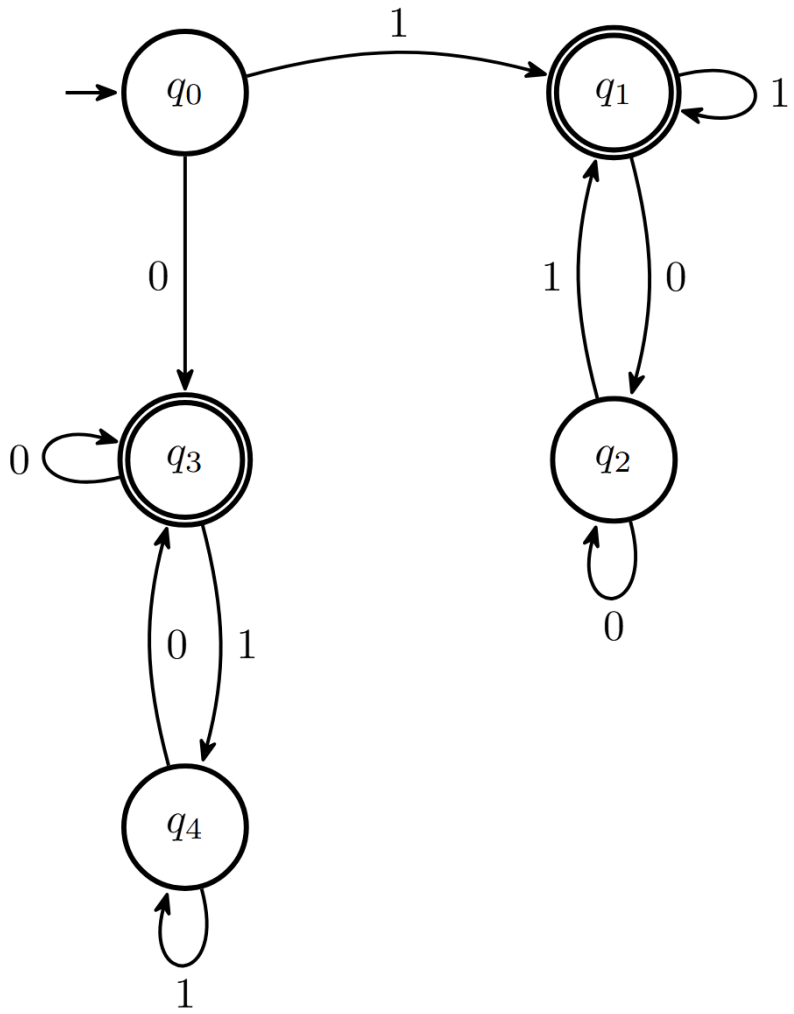
$L = \{w \mid w$ starts with **0** **<u>and</u>** has **odd** length **<u>or</u>** starts with **1** **<u>and</u>** has **even** length$\}$



Examples

Let's practice!

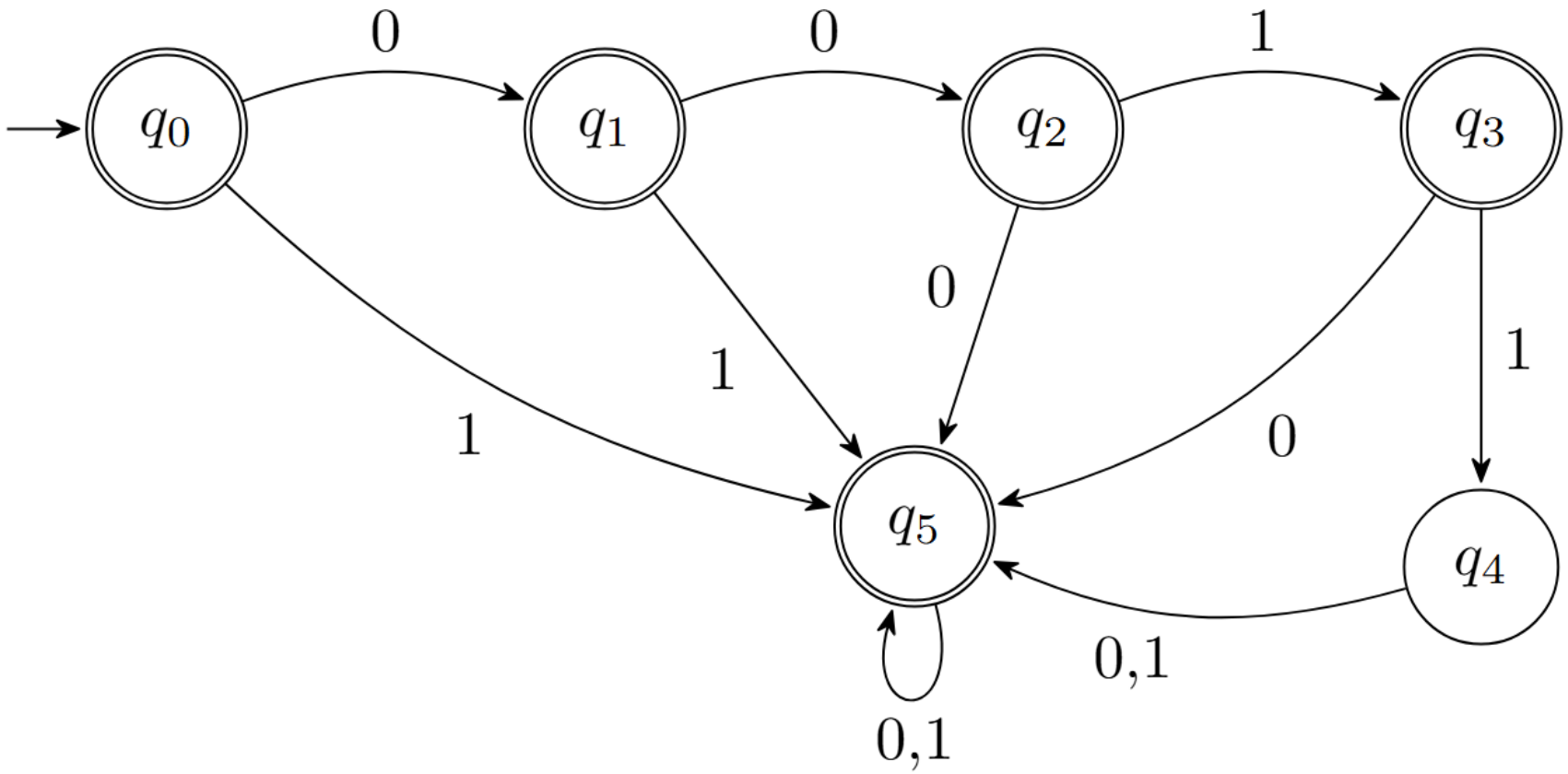
$$L = \{w \mid w \text{ starts and ends with the same symbol}\}$$



Examples

Let's practice!

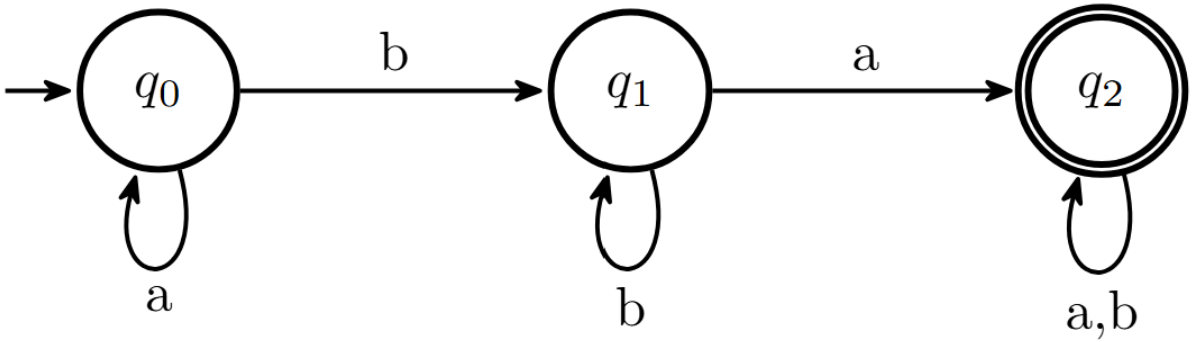
$$L = \{w \mid w \text{ does } \underline{\textbf{not}} \text{ have } \textbf{0011}\}$$



Examples

Let's practice!

$L = \{w \mid w$ is any string **<u>not</u>** in **a*****b***$\}$



Examples

Let's practice!

$$L = \{w \mid w \text{ is a multiple of } \mathbf{3} \text{ in decimal form}\}$$

State Definitions

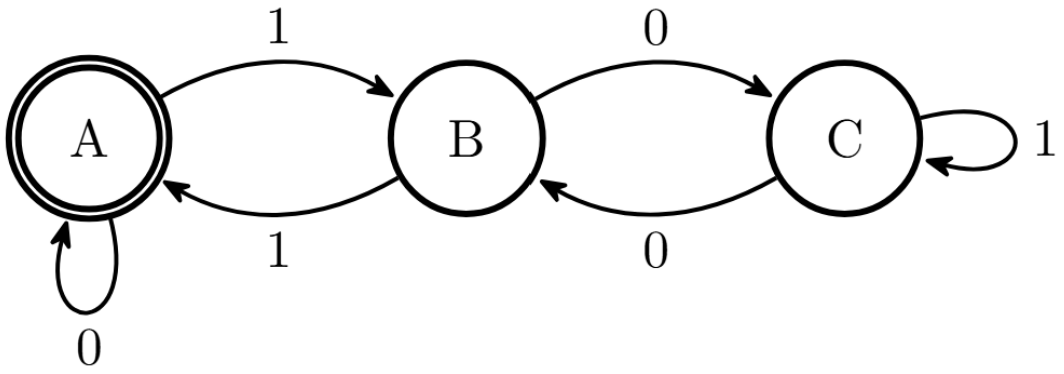
Based on the remainder when dividing by 3:

- $3x + 0 \to$ divisible by 3 (Remainder 0) $\to$ **State A**
- $3x + 1 \to$ not divisible by 3 (Remainder 1) $\to$ **State B**
- $3x + 2 \to$ not divisible by 3 (Remainder 2) $\to$ **State C**

Transition Logic

1. When we see input 0:
(This multiplies the current value by 2: $2 \times \text{Value}$)

$$\begin{aligned}
&\text{From A: } 2(3x) &&= 3(2x) + 0 &&\implies [\mathbf{A} \to \mathbf{A}] \\
&\text{From B: } 2(3x+1) &&= 6x + 2 = 3(2x) + 2 &&\implies [\mathbf{B} \to \mathbf{C}] \\
&\text{From C: } 2(3x+2) &&= 6x + 4 = 3(2x+1) + 1 &&\implies [\mathbf{C} \to \mathbf{B}]
\end{aligned}$$

2. When we see input 1:
(This multiplies the current value by 2 and adds 1: $2 \times \text{Value} + 1$)

$$\begin{aligned}
&\text{From A: } 2(3x) + 1 &&= 3(2x) + 1 &&\implies [\mathbf{A} \to \mathbf{B}] \\
&\text{From B: } 2(3x+1) + 1 &&= 6x + 2 + 1 = 3(2x+1) + 0 &&\implies [\mathbf{B} \to \mathbf{A}] \\
&\text{From C: } 2(3x+2) + 1 &&= 6x + 4 + 1 = 6x + 5 && \\
& &&= 3(2x+1) + 2 &&\implies [\mathbf{C} \to \mathbf{C}]
\end{aligned}$$